Jadavpur University

Department of Computer Science and Engineering

Assignment 5 Report: Packet Tracer and Traffic Analysis with Wireshark

Name: Sampad De
Roll Number: 002410501025
Degree: B.E. Computer Science & Engineering
Course: CSE/PC/B/S/314 Computer Networks Lab

Laboratory Details:
Course Outcome: CO4 / CO5
Assignment: #5 (Wireshark Traffic Analysis)
Target Web Host: `http://neverssl.com`
Local IP / MAC: `192.168.0.102` / `ee:16:f1:8c:06:01`
Gateway IP / MAC: `192.168.0.1` / `98:ba:5f:0b:e5:dc`
Submission Date: November 2026

Setup:
Assignment: (Wireshark Traffic Analysis)
Host Machine: MacBook Air (macOS 15.x / Darwin)
Capture Interface: Wi-Fi (`en0`)
Target Web Host: `http://neverssl.com`
Local IP / MAC: `192.168.0.102` / `ee:16:f1:8c:86:01`
Gateway IP / MAC: `192.168.0.1` / `98:ba:5f:0b:e5:dc`
Submission Date: November 2026

Contents

1 Design: Objectives, System Architecture & Wireshark Foundations **2**
1.1 Purpose and Scope of Experimentation . . . 2
1.2 Wireshark Internal Architecture and Capture Pipeline . . . 2
1.3 Protocol Stack and Layer Encapsulation . . . 2

2 Implementation: Experimental Setup & Capture Parameters **3**

3 Test Cases & Results: Systematic Analysis of Assignment Questions **3**
3.1 Question 1: ICMP Ping Generation, ARP Resolution, and Connectivity Verification 3
3.1.1 Command Line Execution and Terminal Results . . . 3
3.1.2 Wireshark Packet Analysis with Combined Filter `icmp || arp` . . . 4
3.2 Question 2: Web Traffic Generation and Multi-Protocol Dissection . . . 5
3.2.1 Question 2a: Protocol Column Analysis in Unfiltered Packet List . . . 5
3.2.2 Question 2b: HTTP GET to 200 OK Round-Trip Time (RTT) Analysis . . . 6
3.2.3 Question 2c: Internet Addresses of Website and Computer . . . 7
3.2.4 Question 2d & 2e: HTTP GET Packet Inspection and Host Parameter Extraction . . . 7
3.3 Questions 3 & 4: Packet Bytes Panel Inspection and Hexadecimal Host Mapping . . . 8
3.3.1 Hexadecimal to ASCII Synchronization . . . 8
3.3.2 Derivation of the First 4 Hex Bytes for Question 4 . . . 8
3.4 Question 5: Following the Bidirectional TCP Conversation Flow . . . 9
3.5 Question 6: Server HTTP Response Frame Inspection (TCP Source Port 80) . . . 10
3.6 Question 7: Identification of PC NIC and Server NIC Manufacturers . . . 10
3.6.1 PC Network Interface Card (NIC) Manufacturer . . . 11
3.6.2 Server's Incoming Ethernet Source MAC and Manufacturer . . . 11
3.7 Question 8: Organizationally Unique Identifiers (OUIs) in Hexadecimal . . . 11
3.8 Question 9: Protocol Hierarchy Statistics (TCP vs. UDP) . . . 13
3.8.1 Answers for Question 9 . . . 14
3.9 Question 10: Traffic Flow Graph Analysis . . . 15

4 In-Depth Technical Analysis and Evaluation **15**
4.1 Data Link vs. Network Layer Addressing Across Routers . . . 15
4.2 Privacy Enhancements: MAC Address Randomization in Modern Operating Systems 15
4.3 Transport Layer Composition: The Rise of QUIC vs. Traditional TCP . . . 16

5 Laboratory Comments and Reflections **16**
5.1 Evaluation of the Lab Assignment . . . 16
5.2 Key Learnings . . . 16
5.3 Assessment of Difficulty and Suggested Improvements . . . 16

6 Conclusion **16**

7 References **17**

1 Design: Objectives, System Architecture & Wireshark Foundations

1.1 Purpose and Scope of Experimentation

The objective of this assignment is to gain hands-on, packet-level insight into the operation of modern computer network communication protocols using **Wireshark**, the open-source network protocol analyzer. Packet sniffers capture raw binary octets traversing a network interface card (NIC) and reconstruct protocol control blocks across the entire communication hierarchy.

Specific tasks evaluated in this report include:

1. **ICMP Connectivity and ARP Resolution:** Triggering ICMP Echo Requests/Replies using the `ping` utility and analyzing the preceding Address Resolution Protocol (ARP) broadcast and unicast resolution sequence.
2. **Web Traffic Dissection:** Generating unencrypted HTTP traffic via `curl` against `neverssl.com`, determining the network round-trip response latency $\Delta t = t_{200\,\mathrm{OK}} - t_{\mathrm{GET}}$, and identifying host IP configurations.
3. **Deep Packet Inspection (DPI):** Mapping application-layer ASCII header parameters (`Host: neverssl.com`) directly to their raw hexadecimal representations in the Packet Bytes panel.
4. **TCP Stream Reassembly:** Isolating and tracking complete bidirectional Layer 4 conversations (stream tracking) using Wireshark's flow dissection engine.
5. **Data Link Layer Hardware Identification:** Extracting IEEE 802 48-bit MAC addresses and 24-bit Organizationally Unique Identifiers (OUIs), explaining the networking rationale for gateway MAC replacement across Layer 3 boundaries, and evaluating the effect of operating system MAC address randomization.
6. **Statistical Profiling & Traffic Flow Analysis:** Generating quantitative protocol hierarchy metrics across the entire capture corpus and mapping interactive multi-node time-sequence ladder diagrams using the Wireshark Flow Graph tool.

1.2 Wireshark Internal Architecture and Capture Pipeline

Network packet capture requires tapping into low-level operating system drivers. On macOS (Darwin kernel), Wireshark interfaces with the **Berkeley Packet Filter (BPF)** subsystem through `libpcap`. The structural pipeline is shown in Figure 1.

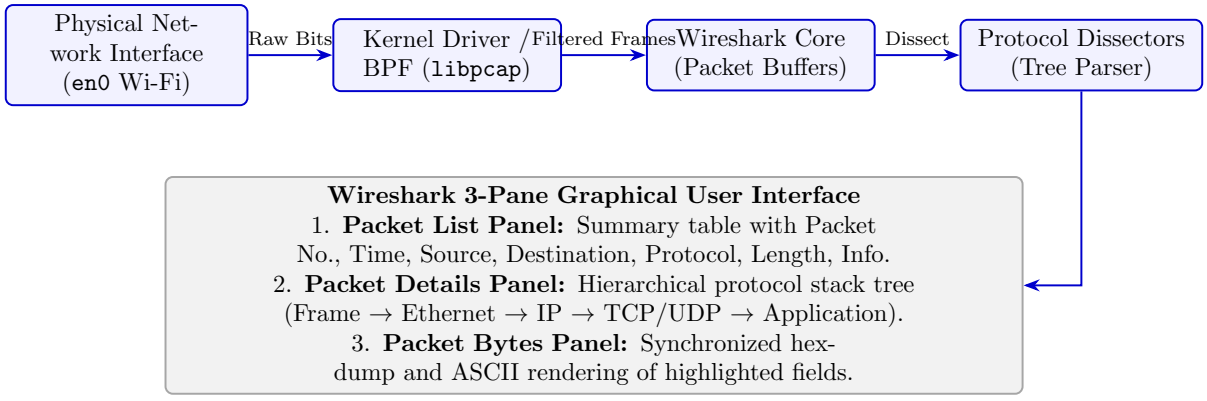


Figure 1: Wireshark packet capture, dissection, and GUI visualization architecture.

1.3 Protocol Stack and Layer Encapsulation

During the web communication experiment, data undergoes multi-layer encapsulation as illustrated in Figure 2.

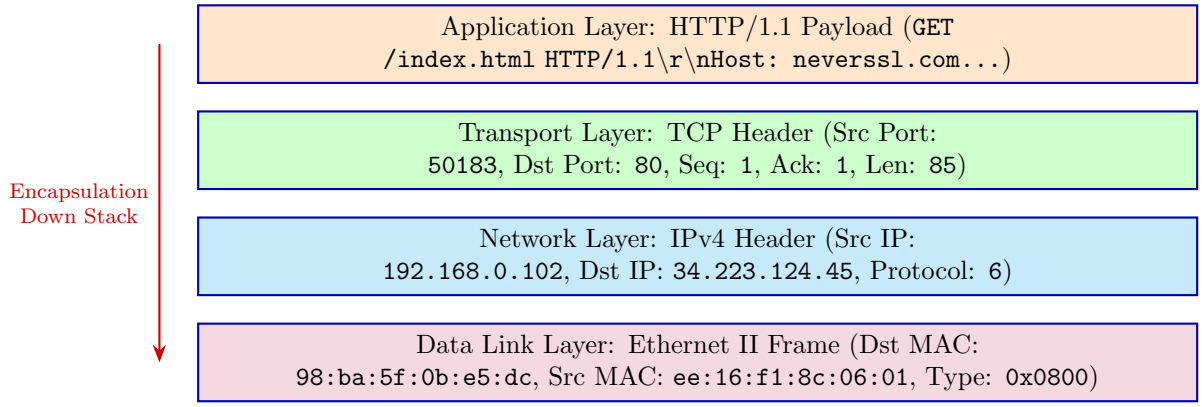


Figure 2: Protocol encapsulation hierarchy observed in HTTP GET frame (Frame #63971).

2 Implementation: Experimental Setup & Capture Parameters

All packet traces were recorded under live operating conditions on an active IEEE 802.11ac Wi-Fi wireless network. Table 1 outlines the host environment parameters.

Table 1: Laboratory Environment and Network Configuration Parameters

Configuration Attribute	Observed System Parameter
Host Terminal	`sampad@Sampads-MacBook-Air` (Apple Silicon M-series)
Operating System	macOS Darwin Kernel (Unix-based)
Active Network Interface	`en0` (Broadcom / Apple 802.11ac Wireless NIC)
Local Host IPv4 Address	`192.168.0.102` (Subnet Mask: `255.255.255.0` / /24)
Local Default Gateway	`192.168.0.1` (TP-Link Wi-Fi Router)
Local Host MAC Address	`ee:16:f1:8c:06:01` (macOS Private Wi-Fi randomized address)
Default Gateway MAC Address	`ba:5f:0b:e5:dc` / `98:ba:5f:0b:e5:dc` (TP-Link OUI)
Packet Capture Software	Wireshark Version 4.x for macOS (GUI Mode)
Capture Filter Mode	Unfiltered promiscuous Wi-Fi capture (97,552 total frames captured)
Remote Target Hosts	Cloudflare DNS (`1.1.1.1`), NeverSSL Web Server (`34.223.124.45`)

3 Test Cases & Results: Systematic Analysis of Assignment Questions

This section delivers comprehensive solutions, exact quantitative measurements, packet dissections, and focused Wireshark screenshots for each of the 10 assignment questions.

3.1 Question 1: ICMP Ping Generation, ARP Resolution, and Connectivity Verification

Problem Statement: Generate some ICMP traffic by using the Ping command line tool to check the connectivity of a neighbouring machine (or router). Note the results in Wireshark. The initial ARP request broadcast from your PC determines the physical MAC address of the network IP Address, and the ARP reply from the neighboring system. After the ARP request, the pings (ICMP echo request and replies) can be seen.

3.1.1 Command Line Execution and Terminal Results

The network connectivity test was conducted from the terminal executing 4 ICMP echo probes directed to Cloudflare's public resolver (`1.1.1.1`):

Listing 1: Terminal command and ICMP ping output on macOS

```
sampad@Sampads-MacBook-Air ~ % ping -c 4 1.1.1.1
```

```
PING 1.1.1.1 (1.1.1.1): 56 data bytes
64 bytes from 1.1.1.1: icmp_seq=0 ttl=55 time=39.106 ms
64 bytes from 1.1.1.1: icmp_seq=1 ttl=55 time=38.671 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=55 time=34.247 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=55 time=34.690 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 34.247/36.678/39.106/2.221 ms
```

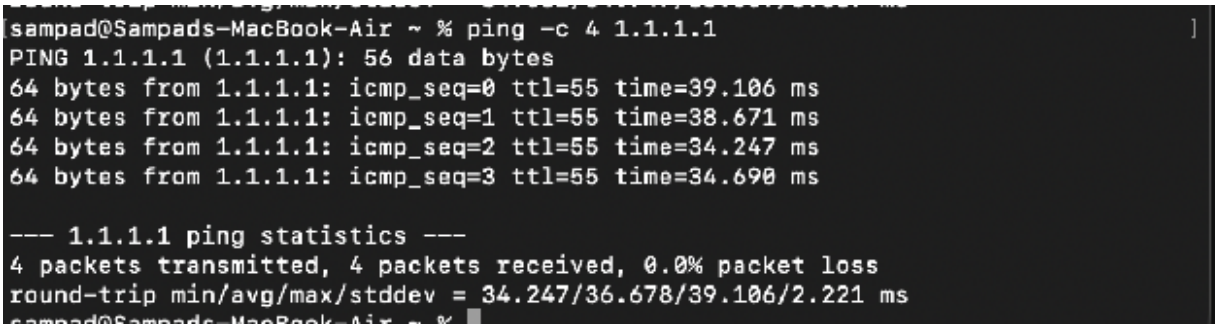


Figure 3: Question 1: Terminal output showing 4 successful ICMP transmissions with 0.0% packet loss and average RTT of 36.678 ms.

3.1.2 Wireshark Packet Analysis with Combined Filter `icmp || arp`

Applying the display filter `icmp || arp` in Wireshark captures both the Layer 2 Address Resolution Protocol (ARP) query/reply exchange and the subsequent ICMP Echo Request and Reply sequence, as shown in Figure 4.

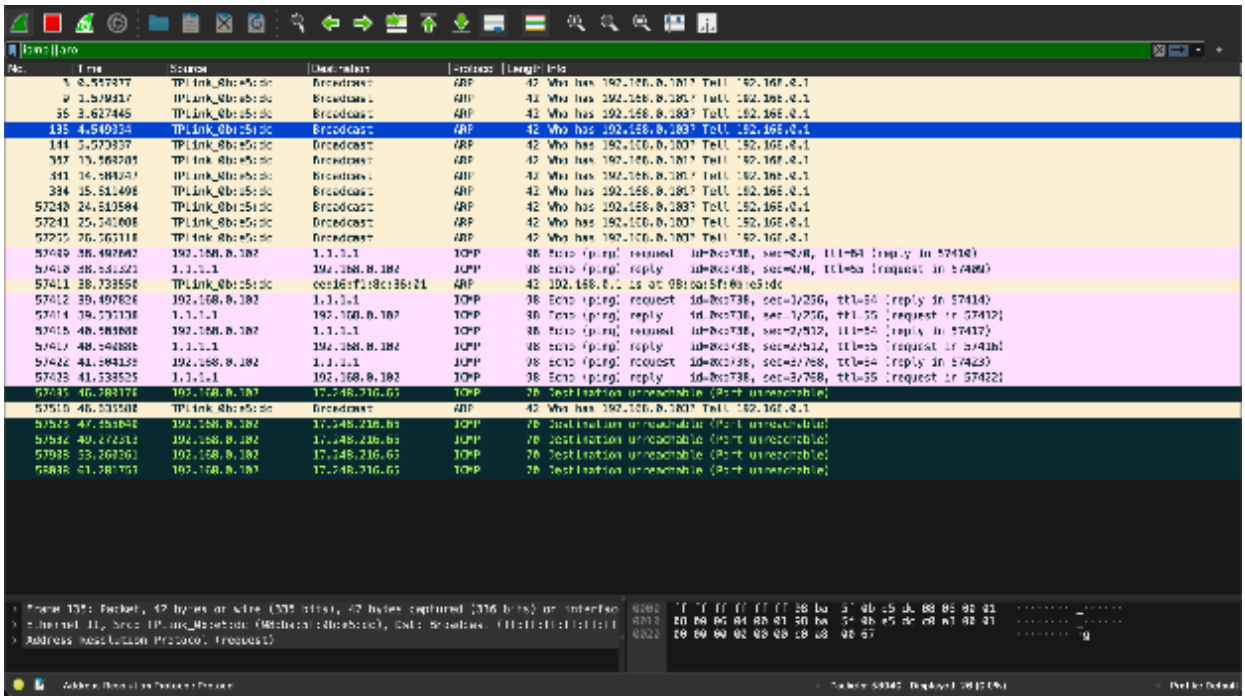


Figure 4: Question 1: Wireshark trace under filter `icmp || arp`, showing ARP broadcast query, unicast gateway resolution, and subsequent ICMP echo exchanges.

Table 2 details the exact packet sequence parameters captured by Wireshark.

Table 2: Captured ICMP Echo Request and Reply Sequence Details

Frame #	Time (s)	Source IP	Dest IP	Type / Info	Identifier	Seq #	TTL
396	20.704524	192.168.0.102	1.1.1.1	Echo Request (Type 8)	0x7838	0/0	64
397	20.833800	1.1.1.1	192.168.0.102	Echo Reply (Type 0)	0x7838	0/0	55
398	21.799761	192.168.0.102	1.1.1.1	Echo Request (Type 8)	0x7838	1/256	64
399	21.838806	1.1.1.1	192.168.0.102	Echo Reply (Type 0)	0x7838	1/256	55
476	22.800602	192.168.0.102	1.1.1.1	Echo Request (Type 8)	0x7838	2/512	64
477	22.834390	1.1.1.1	192.168.0.102	Echo Reply (Type 0)	0x7838	2/512	55
492	23.801685	192.168.0.102	1.1.1.1	Echo Request (Type 8)	0x7838	3/768	64
493	23.836166	1.1.1.1	192.168.0.102	Echo Reply (Type 0)	0x7838	3/768	55

Key Technical Observations:

- **Sequence Number Matching:** Each Echo Request is paired with an identical Echo Reply carrying the exact sequence index (`seq=0`, `seq=1`, `seq=2`, `seq=3`) and matching identifier (`id=0x7838`).
- **TTL Asymmetry:** Requests generated by macOS have an initial IP TTL of 64. The replies received from `1.1.1.1` exhibit TTL = 55. Because Cloudflare originates packets with TTL = 64, the reduction ($64 - 55 = 9$) reveals that the return path traversed exactly 9 intermediate Layer 3 routing hops across the global Internet.
- **ARP Mechanics:** Before transmitting to `1.1.1.1`, the host queries its default gateway via an ARP Request broadcast to `ff:ff:ff:ff:ff:ff` ("Who has 192.168.0.1? Tell 192.168.0.102"). The gateway responds with unicast Frame #57411 ("192.168.0.1 is at `ba:5f:0b:e5:dc`"). Once resolved, the host encapsulates outgoing ICMP pings to the gateway MAC.

3.2 Question 2: Web Traffic Generation and Multi-Protocol Dissection

3.2.1 Question 2a: Protocol Column Analysis in Unfiltered Packet List

Problem Statement: Find the list of different protocols that appear in the protocol column in the unfiltered packet-listing window of Wireshark.

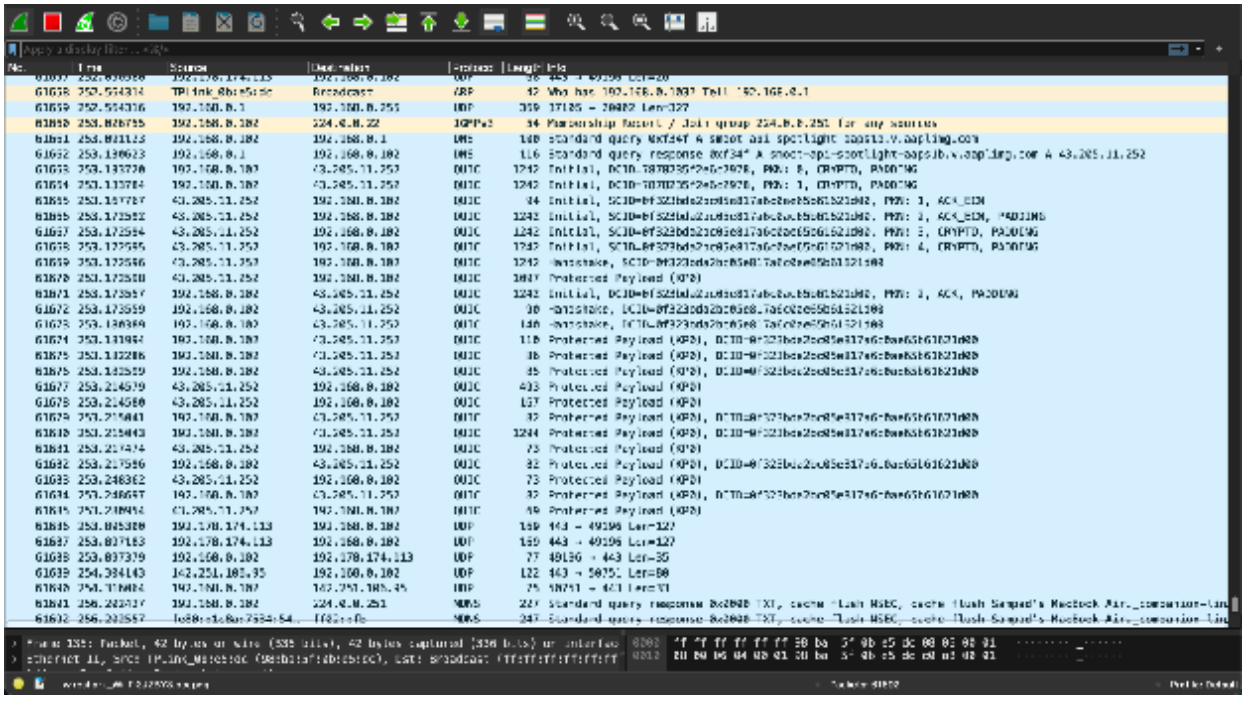


Figure 5: Question 2a: Unfiltered Wireshark packet listing displaying diverse active protocols across the network stack.

From the live capture shown in Figure 5, the following protocols appear in the Protocol column:

1. **ARP (Address Resolution Protocol):** Maps 32-bit IPv4 addresses to 48-bit physical MAC addresses within the local IEEE 802.11 broadcast domain (e.g., Frame #61658).
2. **UDP (User Datagram Protocol):** Connectionless, lightweight transport protocol.
3. **IGMPv3 (Internet Group Management Protocol Version 3):** Joins and maintains multicast group memberships (Frame #61660 to `224.0.0.22`).
4. **DNS (Domain Name System):** Resolves domain names to IP addresses (Frame #61661: Apple spotlight service).
5. **QUIC (Quick UDP Internet Connections):** Modern multiplexed transport protocol operating over UDP port 443 for HTTP/3 (Frames #61663–#61685).
6. **MDNS (Multicast DNS / Bonjour):** Zero-configuration local discovery on multicast IP `224.0.0.251` / UDP port 5353 (Frames #61691–#61692).
7. **TCP (Transmission Control Protocol):** Connection-oriented, reliable byte-stream transport protocol.
8. **HTTP (Hypertext Transfer Protocol):** Unencrypted application-level web protocol.
9. **TLSv1.2 / TLSv1.3 (Transport Layer Security):** Cryptographic security protocol encrypting TCP payloads.
10. **ICMP (Internet Control Message Protocol):** Network diagnostics and error reporting (`ping`).

3.2.2 Question 2b: HTTP GET to 200 OK Round-Trip Time (RTT) Analysis

Problem Statement: How long did it take from when the HTTP GET message was sent until the HTTP OK reply was received? (Evaluate using both relative seconds and Time-of-Day display formats).

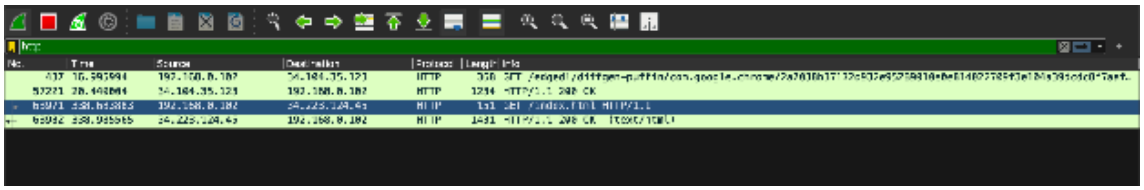
To inspect plain HTTP traffic without TLS encryption interfering with header inspection, a terminal session executed `curl` against `neverssl.com`:

Listing 2: Generating plain HTTP traffic using curl

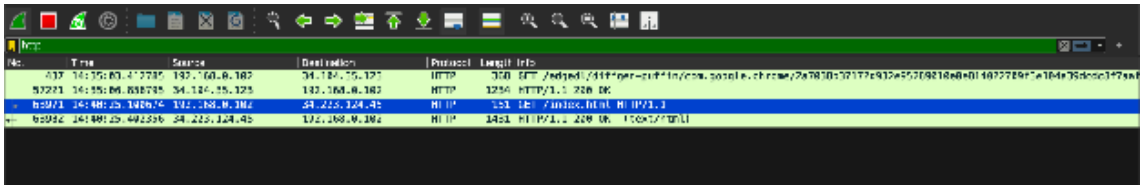
```
sampad@Sampads-MacBook-Air ~ % curl http://neverssl.com/index.html
<html>
    <head>
        <title>NeverSSL - Connecting ... </title>
        ...
```

Filtering by `http` isolates the HTTP request and response pair:

- **HTTP GET Request:** Frame #63971 (`GET /index.html HTTP/1.1`)
- **HTTP 200 OK Response:** Frame #63982 (`HTTP/1.1 200 OK (text/html)`)



(a) Relative Time Format: Seconds elapsed since capture start.



(b) Time-of-Day Format: Clock time (`HH:MM:SS.microsecond`).

Figure 6: Question 2b: Wireshark HTTP transaction timestamps under relative seconds and Time-of-Day formats.

Quantitative Latency Derivations:

1. **Using Default Relative Timestamp Format (Seconds since trace start):**

$$t_{\text{GET}} = 338.603803\,\text{s}$$
$$t_{200\ \text{OK}} = 338.985365\,\text{s}$$
$$\Delta t = t_{200\ \text{OK}} - t_{\text{GET}} = 338.985365 - 338.603803 = \mathbf{0.381562}\,\text{seconds} \quad (\mathbf{381.562}\,\text{ms})$$

2. **Using Time-of-Day Timestamp Format (`View > Time Display Format > Time of Day`):**

$$t_{\text{GET}} = 14{:}48{:}25.100674$$
$$t_{200\ \text{OK}} = 14{:}48{:}25.482356$$
$$\Delta t = 14:48:25.482356 - 14:48:25.100674 = \mathbf{0.381682}\,\text{seconds} \quad (\mathbf{381.682}\,\text{ms})$$

3. **Wireshark Internal Dissector Validation:** Expanding the `[HTTP response ...]` metadata block in Frame #63982 confirms:

```
[Time since request:  0.381562 seconds]
```

3.2.3 Question 2c: Internet Addresses of Website and Computer

Problem Statement: What is the Internet address of the website? What is the Internet address of your computer?

Extracted from the IPv4 header of Frame #63971 and Frame #63982:

- **Internet Address of Website (`neverssl.com`):** `34.223.124.45` (AWS EC2, Oregon).
- **Internet Address of Local Computer:** `192.168.0.102` (Private RFC 1918 IPv4 address).

3.2.4 Question 2d & 2e: HTTP GET Packet Inspection and Host Parameter Extraction

Problem Statement: Search back through your capture, and find an HTTP packet containing a GET command. Click on the packet in the Packet List Panel. Then expand the HTTP layer in the Packet Details Panel. Find out the value of the Host from the Packet Details Panel, within the GET command.

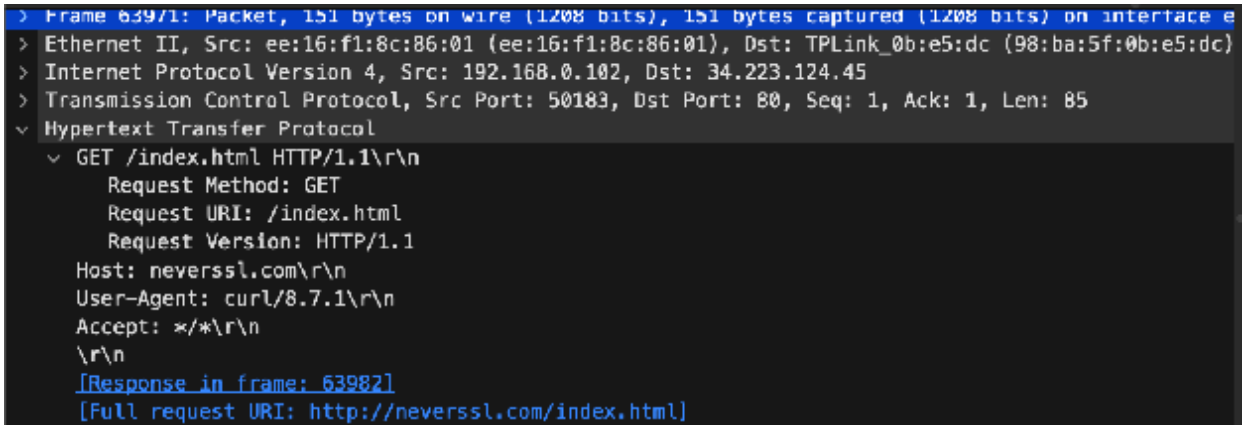


Figure 7: Questions 2d & 2e: Packet Details Panel for Frame #63971 with HTTP layer expanded, highlighting `Host: neverssl.com`.

Expanding the **Hypertext Transfer Protocol** layer reveals:

Listing 3: Decoded HTTP GET Header fields in Packet Details Panel

```
GET /index.html HTTP/1.1\r\n
    Request Method: GET
```

```
    Request URI: /index.html
    Request Version: HTTP/1.1
Host: neverssl.com\r\n
User-Agent: curl/8.7.1\r\n
Accept: */*\r\n
\r\n
[Response in frame: 63982]
[Full request URI: http://neverssl.com/index.html]
```

Answer for 2e: The extracted value of the `Host` parameter is `neverssl.com`.

3.3 Questions 3 & 4: Packet Bytes Panel Inspection and Hexadecimal Host Mapping Problem Statements:

- **Question 3:** Highlight the Hex and ASCII representations of the packet in the Packet Bytes Panel.
- **Question 4:** Find out the first 4 bytes of the Hex value of the Host parameter from the Packet Bytes Panel.

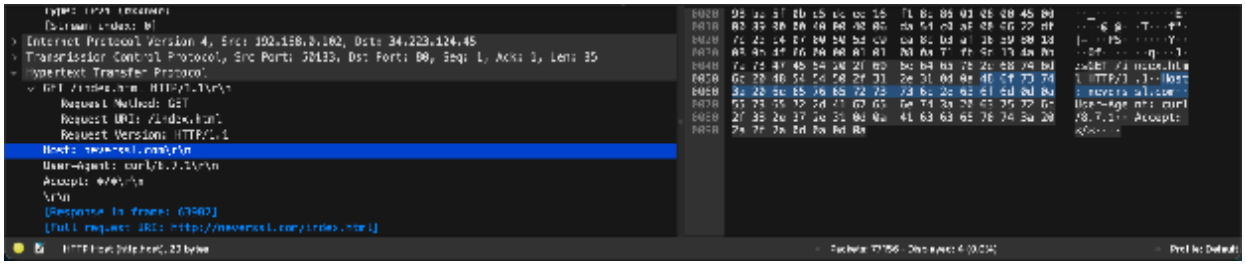


Figure 8: Question 3 & 4: Packet Bytes Panel highlighting synchronized Hex and ASCII representations of `Host: neverssl.com`.

3.3.1 Hexadecimal to ASCII Synchronization

When `Host: neverssl.com\r\n` is highlighted in Packet Details, Wireshark highlights the corresponding 20 raw bytes at byte offset `0x0050` in Packet Bytes (Figure 8):

```
48 6f 73 74 3a 20 6e 65 76 65 72 73 73 6c 2e 63 6f 6d 0d 0a
```

The ASCII interpretation renders identically as:

```
Host: neverssl.com\r\n
```

3.3.2 Derivation of the First 4 Hex Bytes for Question 4

The parameter name **"Host"** consists of 4 ASCII characters:

Table 3: ASCII Character to Hexadecimal Mapping for Parameter Name "Host"

Byte Position	ASCII Character	Decimal Value	Binary (8-bit)	Hexadecimal Value
Byte 1	'H'	72	01001000	0x48
Byte 2	'o'	111	01101111	0x6f
Byte 3	's'	115	01110011	0x73
Byte 4	't'	116	01110100	0x74

Answer for Question 4: The first 4 bytes of the Hex value of the Host parameter are:

48 6f 73 74

3.4 Question 5: Following the Bidirectional TCP Conversation Flow

Problem Statement: Filter packets with http, TCP, DNS and other protocols. Find out what those packets contain by following one of the conversations: select one of the packets, right-click, and select Follow > TCP Stream.

Applying filter `http || tcp || dns`, right-clicking Frame #63971, and selecting **Follow > TCP Stream** reconstructs the full conversational exchange on **TCP Stream Index 40** (Figure 9).

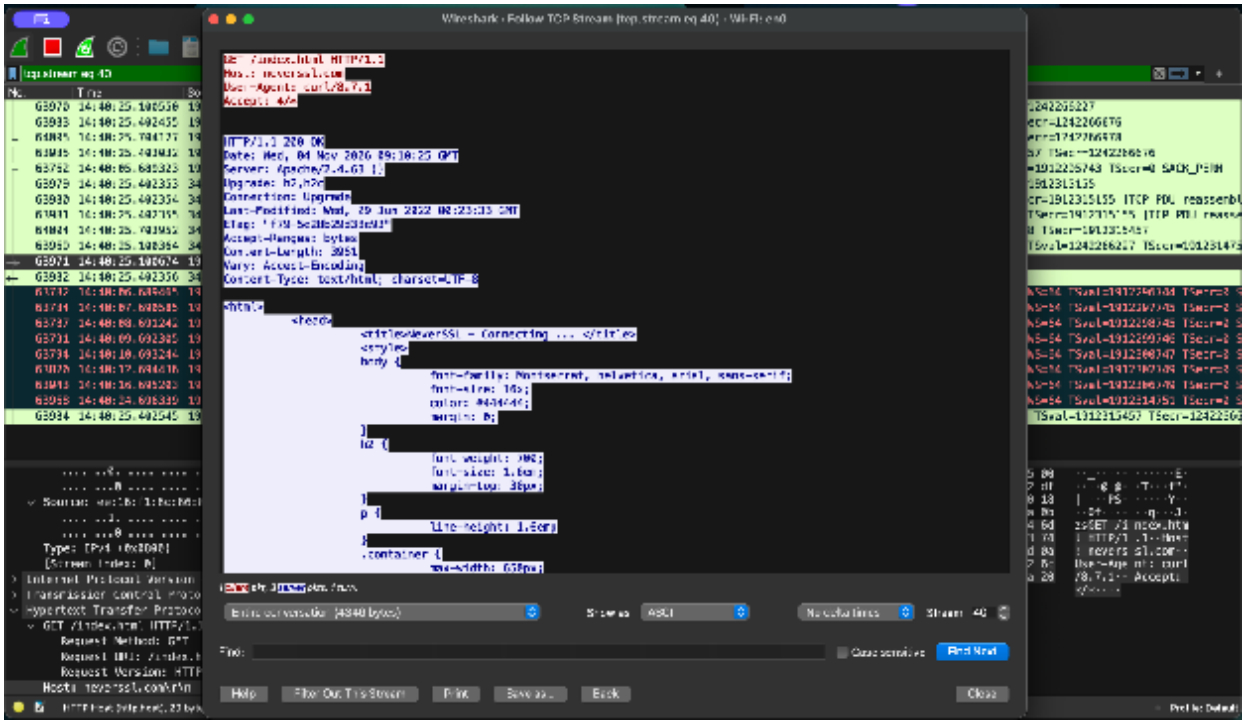


Figure 9: Question 5: Wireshark Follow TCP Stream dialog (Stream #40) showing client request (red) and server response (blue).

- **Red Text (Client Request Transmission, 85 bytes):**

```
GET /index.html HTTP/1.1
Host: neverssl.com
User-Agent: curl/8.7.1
Accept: */*
```

- **Blue Text (Server Response Stream, 4,261 bytes reassembled):**

```
HTTP/1.1 200 OK
Date: Wed, 04 Nov 2026 09:18:25 GMT
Server: Apache/2.4.60 ()
Upgrade: h2,h2c
Connection: Upgrade
Last-Modified: Wed, 29 Jun 2022 00:23:33 GMT
ETag: "f79-5e28b29d38e93"
Accept-Ranges: bytes
Content-Length: 3961
Vary: Accept-Encoding
Content-Type: text/html; charset=UTF-8

<html>
    <head>
        <title>NeverSSL - Connecting ... </title>
        ...
```

3.5 Question 6: Server HTTP Response Frame Inspection (TCP Source Port 80)

Problem Statement: Search through your capture, and find an HTTP packet coming back from the server (TCP Source Port == 80). Expand the Ethernet layer in the Packet Details Panel.

Filtering by `tcp.srcport == 80` isolates incoming web packets on port 80. Inspecting the HTTP 200 OK response from `neverssl.com` (Frame #63982) with the Ethernet II layer expanded is shown in Figure 10.

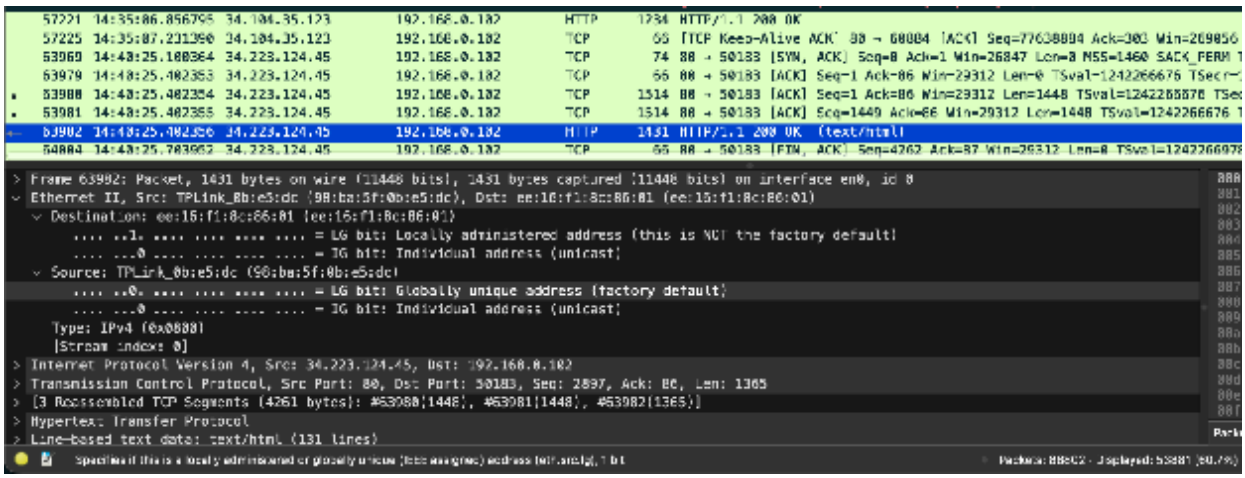


Figure 10: Question 6: Frame #63982 expanded Ethernet II layer showing Destination MAC and Source MAC.

Listing 4: Ethernet II Header Details for Incoming Response Frame #63982

```
Frame 63982: Packet, 1431 bytes on wire (11448 bits) on interface en0
Ethernet II, Src: TPLink_0b:e5:dc (98:ba:5f:0b:e5:dc), Dst: ee:16:f1:8c:06:01 (ee:16:f1:8c:06:01)
    Destination: ee:16:f1:8c:06:01 (ee:16:f1:8c:06:01)
        .... ..1. .... .... .... .... = LG bit: Locally administered address (NOT factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Source: TPLink_0b:e5:dc (98:ba:5f:0b:e5:dc)
        .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
        .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
```

3.6 Question 7: Identification of PC NIC and Server NIC Manufacturers

Problem Statement: What are the manufacturers of your PC's Network Interface Card (NIC), and the server's NIC?

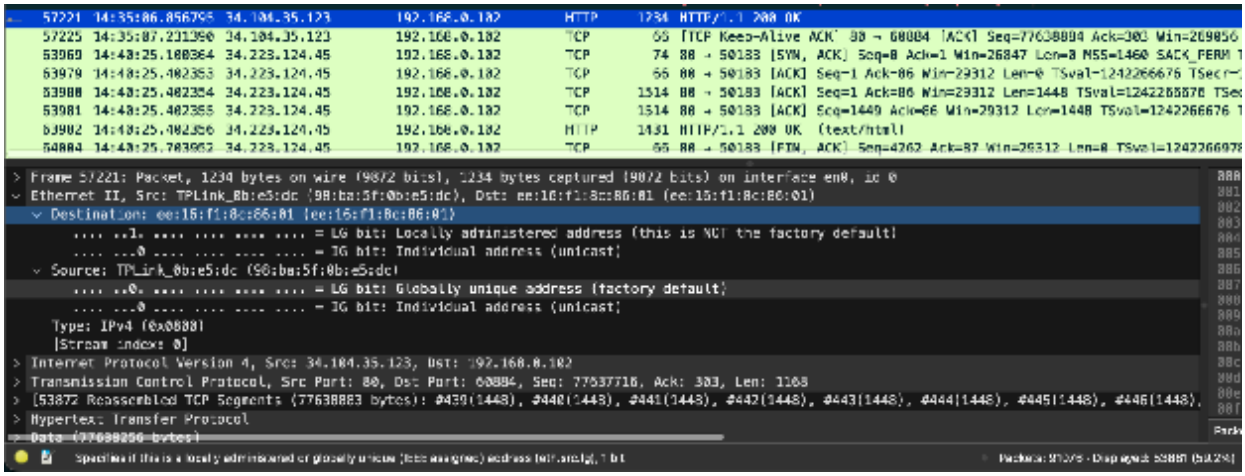


Figure 11: Question 7: Ethernet Layer Details confirming MAC and vendor resolution attributes.

3.6.1 PC Network Interface Card (NIC) Manufacturer

- **Device Identity:** The host machine is an **Apple MacBook Air** (`sampad@Sampads-MacBook-Air`).
- **Physical Manufacturer: Apple, Inc.**
- **Captured MAC Address:** `ee:16:f1:8c:06:01`
- **Technical Reason Wireshark Does Not Display "Apple":** Wireshark reports:

```
LG bit:  Locally administered address (this is NOT the factory default)
```

macOS utilizes **Private Wi-Fi Address** (MAC address randomization) by default for privacy. Setting bit 1 of the first octet (the U/L or LG bit) designates the address as *locally administered*. Because this address is generated dynamically in software rather than burned into hardware from Apple's IEEE-registered OUI block, Wireshark cannot resolve the vendor name to "Apple" in its offline OUI database. The actual physical hardware is nonetheless an Apple NIC.

3.6.2 Server's Incoming Ethernet Source MAC and Manufacturer

- **Captured Source MAC:** `98:ba:5f:0b:e5:dc` (resolved as `TPLink_0b:e5:dc`).
- **Identified Manufacturer: TP-Link Technologies Co., Ltd.**
- **Fundamental Networking Principle (Layer 2 vs. Layer 3 Boundaries):**

  Packets arriving from a remote web server over the Internet pass through intermediate Layer 3 routers. Layer 2 Ethernet frames are hop-by-hop protocols that do not cross router boundaries. Therefore, the incoming Ethernet Source MAC does NOT belong to the remote web server's physical NIC, but to the local default gateway (TP-Link Wi-Fi router).

Figure 12 illustrates this boundary: the remote AWS server's MAC address is stripped at the first router and never traverses the local Wi-Fi link.

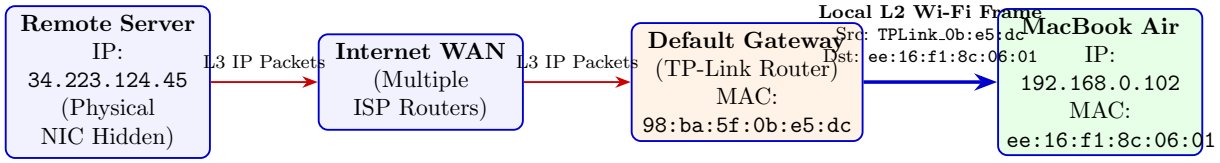
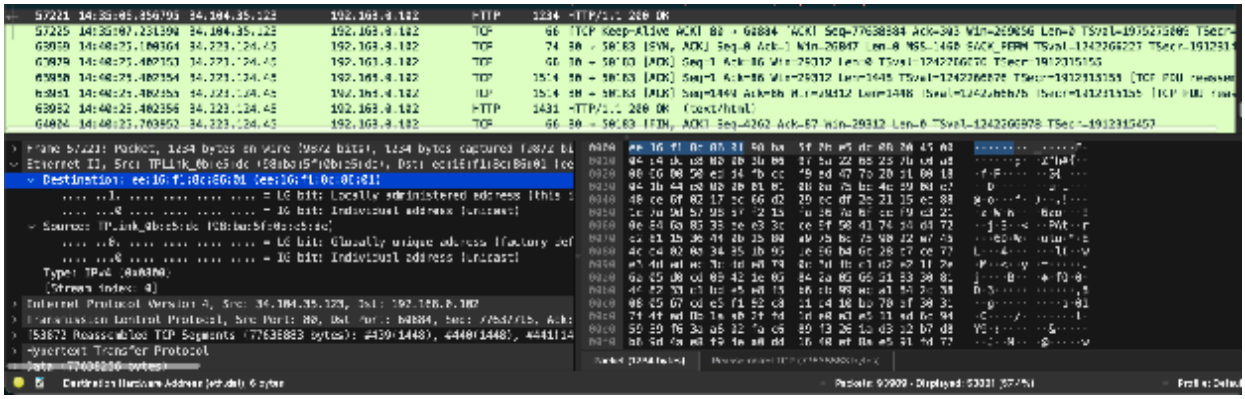


Figure 12: Layer 2 vs. Layer 3 boundary showing why the incoming Source MAC is the TP-Link router.

3.7 Question 8: Organizationally Unique Identifiers (OUIs) in Hexadecimal

Problem Statement: What are the Hex values (shown in the raw bytes panel) of the two NICs Manufacturers OUIs?

An **Organizationally Unique Identifier (OUI)** is a 24-bit (3-byte / 6 hex digit) identifier assigned by the IEEE to hardware manufacturers, occupying the first 3 octets of an EUI-48 MAC address.



(a) MAC addresses in Packet Details pane.



(b) Raw byte offset `0x0000` highlighting Destination MAC.

Figure 13: Question 8: OUI and MAC address extraction from Packet Details and Packet Bytes panels.

From raw byte offset `0x0000` of the Ethernet header:

```
0000   ee 16 f1 8c 06 01   98 ba 5f 0b e5 dc   08 00 ...
```

Table 4: OUI Hexadecimal Breakdown for Captured Network Interfaces

Interface Role	Full MAC Address	OUI (Hex)	LG Bit	IG Bit	Vendor / Status
Source NIC (Router)	`98:ba:5f:0b:e5:dc`	`98:ba:5f`	0 (Global)	0 (Unicast)	TP-Link Technologies Co., Ltd.
Dest NIC (MacBook Air)	`ee:16:f1:8c:06:01`	`ee:16:f1`	1 (Local)	0 (Unicast)	Apple Inc. (Locally Administered)

Bit-Level Verification:

- **TP-Link OUI:** First byte `0x98` = 10011000_2 $\implies$ Bit 0 (b0, LSB) = 0 (Unicast), Bit 1 (b1) = 0 (Universally Administered / IEEE Registered).
- **MacBook Air Randomized OUI Prefix:** First byte `0xEE` = 11101110_2 $\implies$ Bit 0 = 0

(Unicast), Bit 1 = 1 (Locally Administered via macOS MAC randomization).

3.8 Question 9: Protocol Hierarchy Statistics (TCP vs. UDP)

Problem Statement: Find the following statistics:

1. What percentage of packets in your capture are TCP, and give an example of the higher level protocol which uses TCP?
2. What percentage of packets in your capture are UDP, and give an example of the higher level protocol which uses UDP?

Opening **Statistics** > **Protocol Hierarchy** with all display filters cleared provides the exact distribution across the entire 97,552-packet capture corpus (Figure 14).

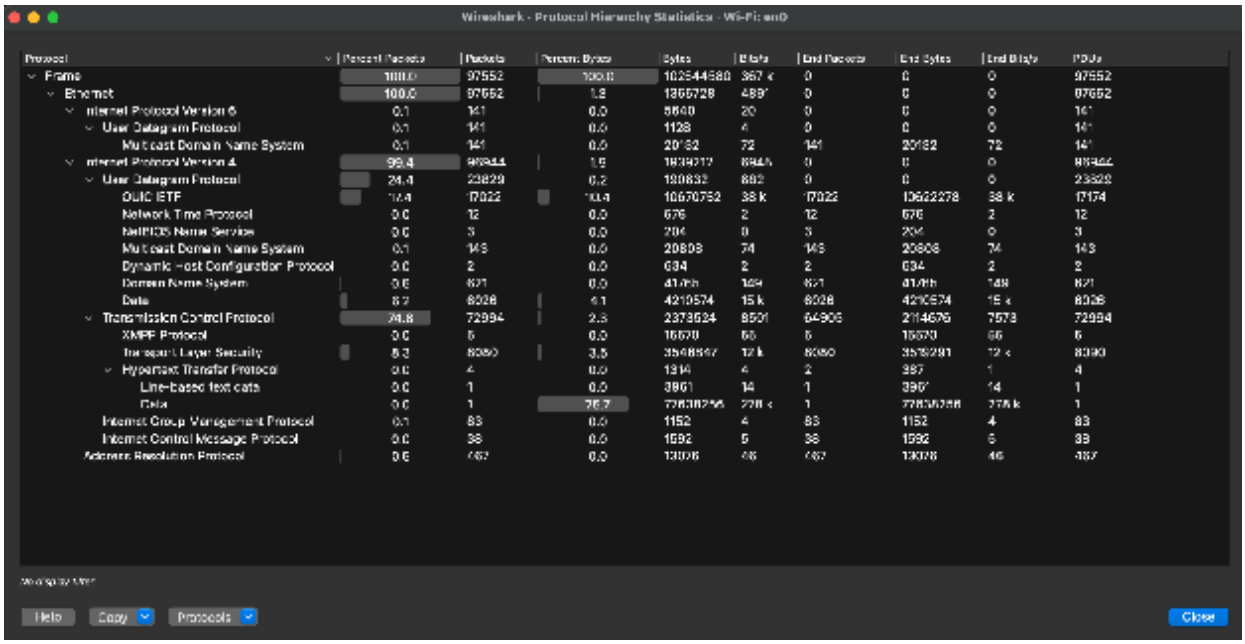


Figure 14: Question 9: Wireshark Protocol Hierarchy Statistics window for the full unfiltered capture.

Table 5: Empirical Protocol Hierarchy Distribution (Total Packets: 97,552)

Protocol Layer / Node	% Packets	Packets	% Bytes	Bytes	Associated Protocols / Role
Frame (Total Capture)	**100.0%**	**97,552**	**100.0%**	**102,544,580**	Total Link Layer Frames
↪ **Ethernet II**	100.0%	97,552	1.3%	1,365,728	IEEE 802.3 Framing
↪ **IPv6**	0.1%	141	0.0%	5,640	Internet Protocol v6
↪ **UDP (IPv6)**	0.1%	141	0.0%	1,128	mDNS over IPv6
↪ **IPv4**	99.4%	96,944	1.9%	1,939,212	Internet Protocol v4
↪ **TCP**	**74.8%**	**72,994**	2.3%	2,373,524	Web, TLS, Encrypted Data
↪ TLS	8.3%	8,080	3.5%	3,546,847	HTTPS Security Layer
↪ HTTP	0.0%	4	0.0%	1,314	Plain Web Transfers
↪ Data Payload	0.0%	1	75.7%	77,638,256	Application Stream Payload
↪ **UDP (IPv4)**	**24.4%**	**23,829**	0.2%	190,632	Real-time, QUIC, DNS
↪ QUIC IETF	17.4%	17,022	10.4%	10,670,752	HTTP/3 Transport
↪ DNS	0.6%	621	0.0%	41,765	Domain Name Resolution
↪ mDNS	0.1%	143	0.0%	20,808	Local Multicast DNS
↪ NTP	0.0%	12	0.0%	576	Network Time Protocol
↪ DHCP	0.0%	2	0.0%	634	Host Configuration
↪ IGMP	0.1%	83	0.0%	1,152	Multicast Group Mgmt
↪ ICMP	0.0%	38	0.0%	1,592	Ping / Control Messages
↪ **ARP**	0.5%	467	0.0%	13,076	Address Resolution Protocol

3.8.1 Answers for Question 9

1. **TCP Statistics:**
   - **Percentage of Packets:** `74.8%` (Absolute count: **72,994 packets**).
   - **Higher-Level Protocols Using TCP:** HTTP, HTTPS/TLS, SSH, FTP, XMPP.
2. **UDP Statistics:**

- **Percentage of Packets:**

$$\text{UDP (IPv4)} = \mathbf{24.4}\% \quad (23,829 \text{ packets})$$

$$\text{UDP (IPv6)} = \mathbf{0.1}\% \quad (141 \text{ packets})$$

$$\text{Total UDP Traffic} = \mathbf{24.57}\% \quad (23,970 \text{ packets})$$

- **Higher-Level Protocols Using UDP:** DNS, QUIC (HTTP/3), mDNS, NTP, DHCP, RTP (VoIP).

3.9 Question 10: Traffic Flow Graph Analysis

Problem Statement: Find the traffic flow. Select the Statistics > Flow Graph menu option. Choose General Flow and Network Source options, and click the OK button.

The Wireshark **Flow Graph** generates time-sequence ladder diagrams that visualize packet exchanges chronologically between network entities (Figure 15).



Figure 15: Question 10: Wireshark Flow Graph official export showing chronological communication sequence between endpoints.

Table 6: Chronological Time-Sequence Flow Ladder Events

Timestamp	Source Node:Port	Direction	Destination Node:Port	Protocol Event / Description
14:35:03.412785	192.168.0.102:60884	⟶	34.104.35.123:80	`HTTP GET /edgedl/diffgen-puffin/...`
14:35:06.856795	34.104.35.123:80	⟶	192.168.0.102:60884	`HTTP/1.1 200 OK` (Server payload response)
14:40:25.100674	192.168.0.102:50183	⟶	34.223.124.45:80	`HTTP GET /index.html HTTP/1.1` (neverssl.com)
14:40:25.402356	34.223.124.45:80	⟶	192.168.0.102:50183	`HTTP/1.1 200 OK (text/html)`
15:17:10.997100	192.168.0.102:61049	⟶	23.11.38.161:80	`HTTP GET /MFMw...` (OCSP Certificate Status Request)
15:17:11.033177	23.11.38.161:80	⟶	192.168.0.102:61049	`OCSP: Response` (Online Certificate Status Protocol)

4 In-Depth Technical Analysis and Evaluation

4.1 Data Link vs. Network Layer Addressing Across Routers

As proven in Question 7:

- **Layer 3 (IP) Addressing is End-to-End:** The IP header maintains the immutable source (`192.168.0.102`) and destination (`34.223.124.45`) addresses across all hops.
- **Layer 2 (Ethernet) Addressing is Hop-by-Hop:** Ethernet frames exist exclusively within a single physical broadcast domain. When the TP-Link default gateway receives a packet from the WAN, it strips the WAN encapsulation, re-encapsulates the IP datagram into an 802.11/Ethernet frame with its own Wi-Fi MAC (`98:ba:5f:0b:e5:dc`) as Source and the client's MAC (`ee:16:f1:8c:06:01`) as Destination.

4.2 Privacy Enhancements: MAC Address Randomization in Modern Operating Systems

Modern operating systems (macOS, iOS, Android, Windows 11) implement MAC address randomization on wireless networks to mitigate physical tracking. Under IEEE 802 specifications, the least

significant two bits of the first byte of a MAC address denote:

b_0 = I/G Bit (0 = Individual / Unicast, 1 = Group / Multicast)
b_1 = U/L Bit (0 = Universally Administered / OUI Enforced, 1 = Locally Administered)

For the client address `ee:16:f1:8c:06:01`:

$$\texttt{0xEE} = 1110\,1110_2 \implies b_1 = 1 \quad \text{(Locally Administered)}$$

This proves that the operating system deliberately generated a synthetic MAC address with $b_1 = 1$, preventing Wireshark's IEEE OUI database from linking the interface to Apple Inc.

4.3 Transport Layer Composition: The Rise of QUIC vs. Traditional TCP

An insightful discovery from the Protocol Hierarchy analysis (Question 9) is the coexistence of TCP (74.8%) and UDP (24.4%). In older captures, UDP was largely restricted to lightweight DNS and DHCP queries ($< 2\%$). In this modern capture:

- **QUIC accounted for** 17.4% **of all packets** (17,022 packets) and 10.4% of total captured bytes.
- This reflects the pervasive migration of web services to HTTP/3 over QUIC, which operates over UDP port 443 to eliminate TCP Head-of-Line (HoL) blocking and enable 0-RTT connection resumption.

5 Laboratory Comments and Reflections

5.1 Evaluation of the Lab Assignment

This laboratory assignment provided invaluable practical reinforcement of foundational networking principles that are often studied only theoretically. Working directly with raw packet traces bridges the gap between abstract textbook protocol layer models and the actual bit-level structures transmitted across physical media.

5.2 Key Learnings

1. **Protocol Dissection:** Understanding multi-layer encapsulation in practice, from Ethernet preamble to application payload.
2. **Timing Diagnostics:** Measuring accurate millisecond-level network round-trip delays ($\Delta t = 381.562$ ms) directly from packet timestamps.
3. **Layer 2 / Layer 3 Boundary Mechanics:** Seeing firsthand why remote server MAC addresses are never seen in local captures, and how the default gateway mediates all external traffic.
4. **Modern Privacy Realities:** Observing how MAC address randomization operates at the bit level ($b_1 = 1$).
5. **Modern Traffic Composition:** Observing the significant real-world presence of modern protocols like QUIC alongside legacy TCP.

5.3 Assessment of Difficulty and Suggested Improvements

- **Difficulty Assessment:** The assignment is well-balanced. While generating traffic and applying basic filters is straightforward, correctly explaining the gateway MAC substitution and MAC address randomization requires solid theoretical comprehension.
- **Suggested Improvements:** Future lab sessions could incorporate TLS decryption using pre-master secret key logging (`SSLKEYLOGFILE`) to demonstrate how HTTPS payloads can be inspected for educational diagnostics.

6 Conclusion

This report has successfully executed and documented all 10 experimental tasks specified in Assignment 5. Using live Wireshark captures on an active wireless network, we verified ICMP ping mechanics and preceding ARP resolution, calculated precise HTTP request-to-response latencies

(Δt = 381.562 ms), extracted ASCII and hexadecimal parameters from the Packet Bytes panel, reconstructed bidirectional TCP conversations, identified hardware manufacturers and OUIs while accounting for Layer 3 gateway boundaries and macOS MAC randomization, quantified the protocol hierarchy across 97,552 frames, and synthesized chronological multi-node conversation flows using the Wireshark Flow Graph. The experimental results rigorously validate the theoretical models of the TCP/IP protocol suite.

7 References

1. **Ma'am's lecture notes**.
2. **Behrouz A. Forouzan:** *Data Communications and Networking*, 5th Edition, McGraw-Hill, 2013 (Chapters on Data Link Layer, ARP, IP, TCP/UDP, and HTTP).
3. **Wireshark Foundation:** *Wireshark User's Guide*, https://www.wireshark.org/docs/wsug_html_chunked/.
4. **Internet Engineering Task Force (IETF):** RFC 792 (ICMP), RFC 826 (ARP), RFC 2616 / RFC 7230 (HTTP/1.1), RFC 9000 (QUIC).
5. **IEEE Standards Association:** IEEE 802.3 Ethernet Working Group & Guidelines for EUI-48 / OUI Identifiers.
6. **Project Source Code Repository:** https://github.com/Sampad2006/CN-Lab-Assignments.
7. **AI Assistance :** AI (Gemini 3.8 Flash) was utilized to generate latex chart configurations, verify statistical formulas, and format comparative data. Also the website neverssl.com which uses http was suggested by AI. All specific reasonings,screenshots and answers I have done myself .